CO3133 · DEEP LEARNING AND ITS APPLICATIONS · SEMESTER-261

Assignment 1 · Foundations of Deep Learning Pipelines and Architectures

From Linear Models to Transformers on Fashion-MNIST

A controlled comparison of five architecture families under one split, one protocol

GROUP RTX5090

Dang Thanh Phu · 2452967

Cao Duc Thanh · 2453164

INSTRUCTOR

Dr. Le Thanh Sach

HCMUT – VNU-HCM · Faculty of CSE

AGENDA

Two presenters, one storyline

PART A · CAO DUC THANH

1 · Problem formulation

2 · Dataset, EDA, split & leakage control

3 · End-to-end PyTorch pipeline

4 · Linear, MLP and CNN

5 · LSTM and Vision Transformer

PART B · DANG THANH PHU

6 · Experimental design & main results

7 · Training behaviour, trade-offs, ablations

8 · Error analysis & qualitative examples

9 · Live demo, links & AI disclosure

10 · Conclusion

1 · PROBLEM FORMULATION

One grayscale image in, one of ten clothing classes out

INPUT

`x : [1, 28, 28]`

pixels 0–255 → normalized

MODEL

`z = f(x) : [10]`

raw logits, no softmax

OUTPUT

`ŷ = argmax z`

class id 0–9, one per image

```
Loss: L = -log softmax(z)[y]  →  nn.CrossEntropyLoss on logits (LogSoftmax + NLL inside)
```

Look-alike classes

Shirt, T-shirt, Pullover, Coat share one silhouette at 28×28

No colour cue

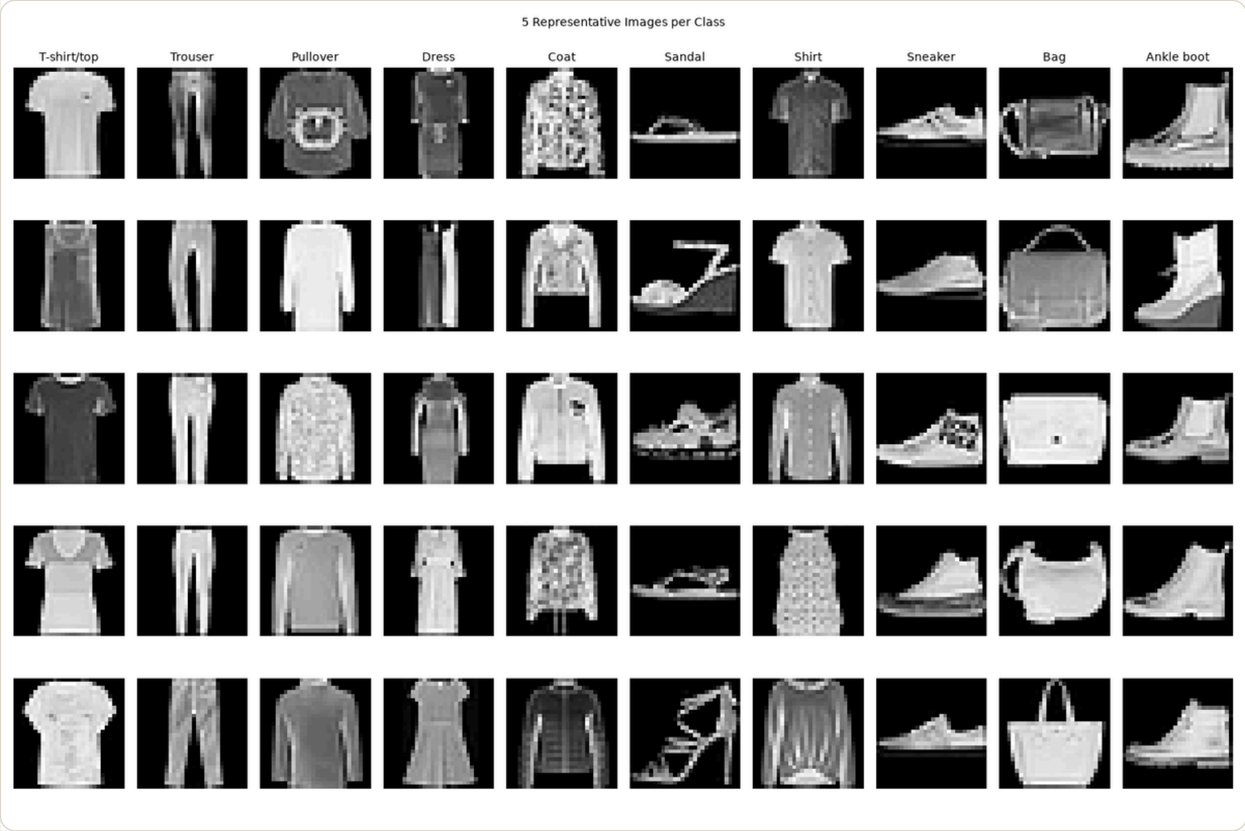
Only shape, edges and texture carry the class

Representation choice

Flatten, 2-D grid, row sequence or patches — each keeps different structure

2 · DATASET & EDA

Balanced classes, uneven difficulty



70,000

grayscale 28×28 images · 10 classes · Zalando Research, MIT licence, IDX ≈ 30 MB

6,000 / 1,000

images per class in official train / test → imbalance ratio 1.0, so accuracy and macro-F1 are not skewed

> **EDA reading:** footwear, trousers and bags are easy; T-shirt, Pullover, Coat and Shirt look alike → we expected Shirt to be the hardest class.

Main comparison and debugging both on Fashion-MNIST (1-batch smoke runs); MNIST and CIFAR-10 not used.

2 · SPLIT, PREPROCESSING & LEAKAGE CONTROL

One fixed split, reused by every model



Official 60k training set → random 90 / 10 split (torch.randperm, seed 666, not stratified) · official 10k test set kept untouched

Same split everywhere

Indices saved once to results/split.npz and reloaded by every run, so all five models see identical train / val / test partitions.

No statistics leak

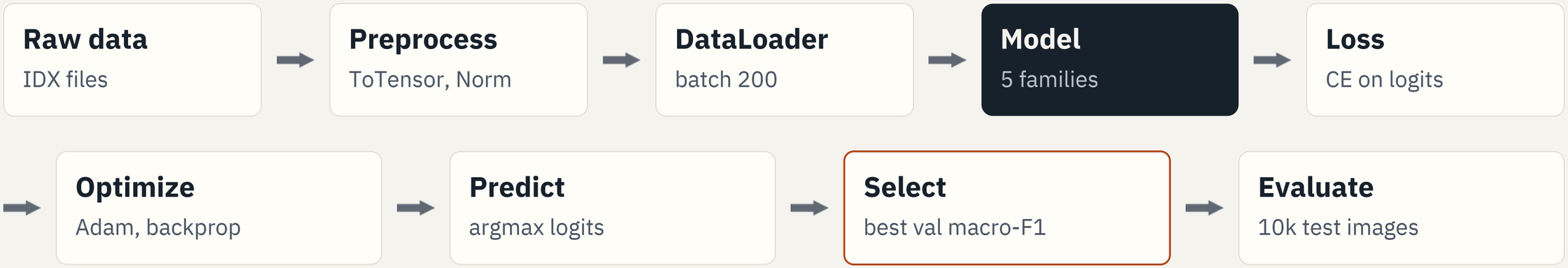
Normalization uses μ = 0.2859, σ = 0.3529 computed on the 54k training images only, then applied unchanged to val and test.

One batch, checked

ToTensor → Normalize, no augmentation. A batch is images [200, 1, 28, 28] float32 + labels [200] int64; only train is shuffled.

3 · PIPELINE & TRAINING SETUP

One pipeline shared by all five models



Optimizer Adam, lr 1e-3	Epochs · batch 10 · 200 (ViT also 30)	Regularization Dropout only	Scheduler · early stop None · keep best
Seed 666 (split + init)	Hardware RTX 5070 Ti, FP32	Software Python 3.12.9, PyTorch 2.11 + CUDA 12.8	Self-implemented All models + loops

4 · MODELS 1–3

From flat vectors to spatial feature maps

FLATTENED 784

Linear / softmax

`Flatten → Linear(784, 10)`

One hyperplane per class. Cannot model interactions between pixels, so it underfits.

7,850

parameters

FLATTENED 784

MLP

`Linear(784, 256) → ReLU → Dropout 0.2 → Linear(256, 10)`

ReLU gives non-linear boundaries; dropout limits co-adaptation. Pixel order still does not matter.

203,530

parameters

2-D GRID 1×28×28

CNN (own design)

`Conv3×3(32) → Conv3×3(128) → Conv3×3(128) → MaxPool 2 → FC 128 → Dropout 0.35 → FC 10`

Shared 3×3 filters find edges and seams anywhere; pooling halves 28×28 into 128 feature maps of 14×14.

3,397,578

parameters · 94.5% in the first FC, convs only 185k

Reading an image as a sequence

LSTM · rows

17,043,466 params

Timestep

t = one pixel row, top → bottom, T = 28

Input size

28 pixels per step → tensor [B, 28, 28]

Hidden representation

ht, ct of size 2048, updated by input, forget, cell and output gates

Classifier

Linear(2048, 10) on the last state h28

ViT · patches

28,202 params

Tokens & embedding

49 patches of 4×4 → Conv2d(1, 32, k = 4, s = 4) → [B, 49, 32]

Positional encoding

+ learnable [CLS] token, + learnable position table [1, 50, 32]

Attention (3 pre-norm blocks, 4 heads)

Q = K = V = LayerNorm(X) : [B, 50, 32] → 50×50 weights per head → [B, 50, 32]

Classifier

MLP 32→64→32 (GELU) per block; Linear(32, 10) on [CLS]

6 · EXPERIMENTAL DESIGN

Three questions, one changed factor each

Fixed in every run: split & seed 666 · preprocessing · Adam 1e-3 · batch 200 · 10 epochs · best-val-macro-F1 checkpoint · metrics = test accuracy, macro-F1, parameters, training and inference time

Experiment	Changed factor	Hypothesis → decision criterion
EXP-1 Architecture Comparison	Model architecture: Linear vs MLP vs CNN vs LSTM vs ViT	architectures capable of learning spatial or sequential relationships would perform better than simpler → yes, spatial learning model like CNN is the best but more parameter is not always stronger in some case.
EXP-2 Attention vs convolution	4×4 patches + self-attention vs CNN	no locality prior is worse → ViT below CNN after 10 epochs, improves with more epochs
EXP-3 Training duration	ViT trained for 10 vs 30 epochs, keeping architecture and other settings fixed.	more train time for ViT is better → Vit with 30 epochs is improved

6 · MAIN RESULTS · 10,000 TEST IMAGES

CNN leads on quality

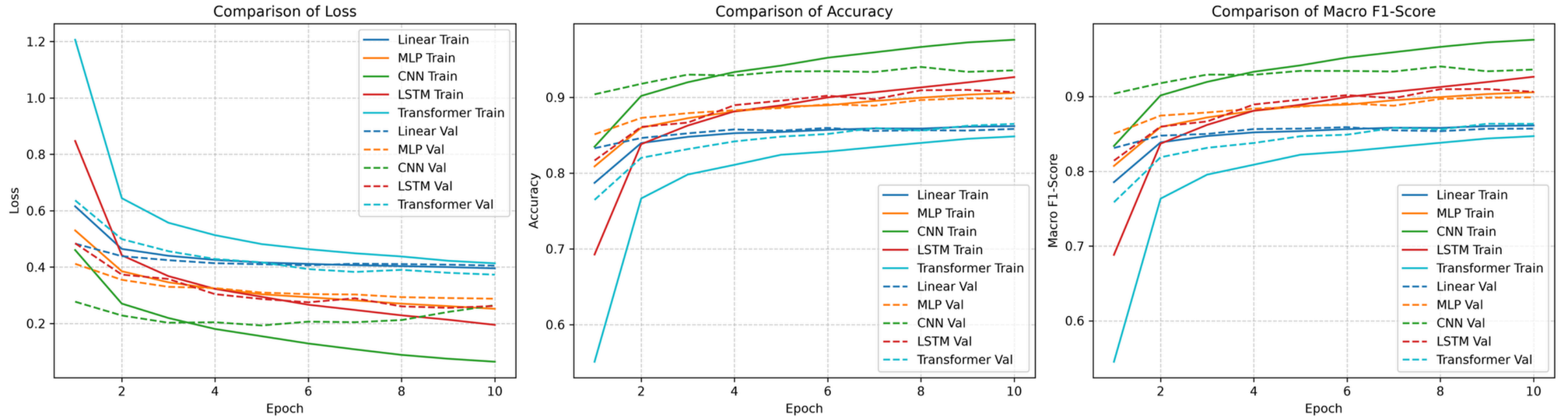
Model	Params	Best ep.	Test acc	Macro-F1	Train	Inference
Linear	7,850	6	0.8391	0.8371	58.9 s	0.73 s
MLP	203,530	10	0.8812	0.8816	92.2 s	0.76 s
CNN · best	3,397,578	8	0.9319	0.9316	98.8 s	0.78 s
LSTM (h 2048)	17,043,466	9	0.9004	0.9004	185.5 s	2.03 s
ViT · 10 epochs	28,202	9	0.8481	0.8492	75.2 s	0.79 s
ViT · 30 epochs	28,202	30	0.8733	0.8738	222.5 s	0.78 s

Train = wall-clock for all epochs incl. per-epoch evaluation and checkpointing. Inference = one pass over the 10k test set incl. data loading, so gaps under 0.1 s are loader noise; only the LSTM is clearly slower (2.6× CNN). Single run per model, seed 666.

TIME IS NOT ALWAYS CORRECT, IT MAY CHANGE BASE ON DIFFERENT GPU

7 · TRAINING BEHAVIOUR · SOLID = TRAIN, DASHED = VAL

Three different learning regimes



CNN · mild overfit

Val loss bottoms at 0.193 (ep 5), rises to 0.265 while train loss hits 0.064. Checkpoint at ep 8 avoids the tail.

Linear · underfit

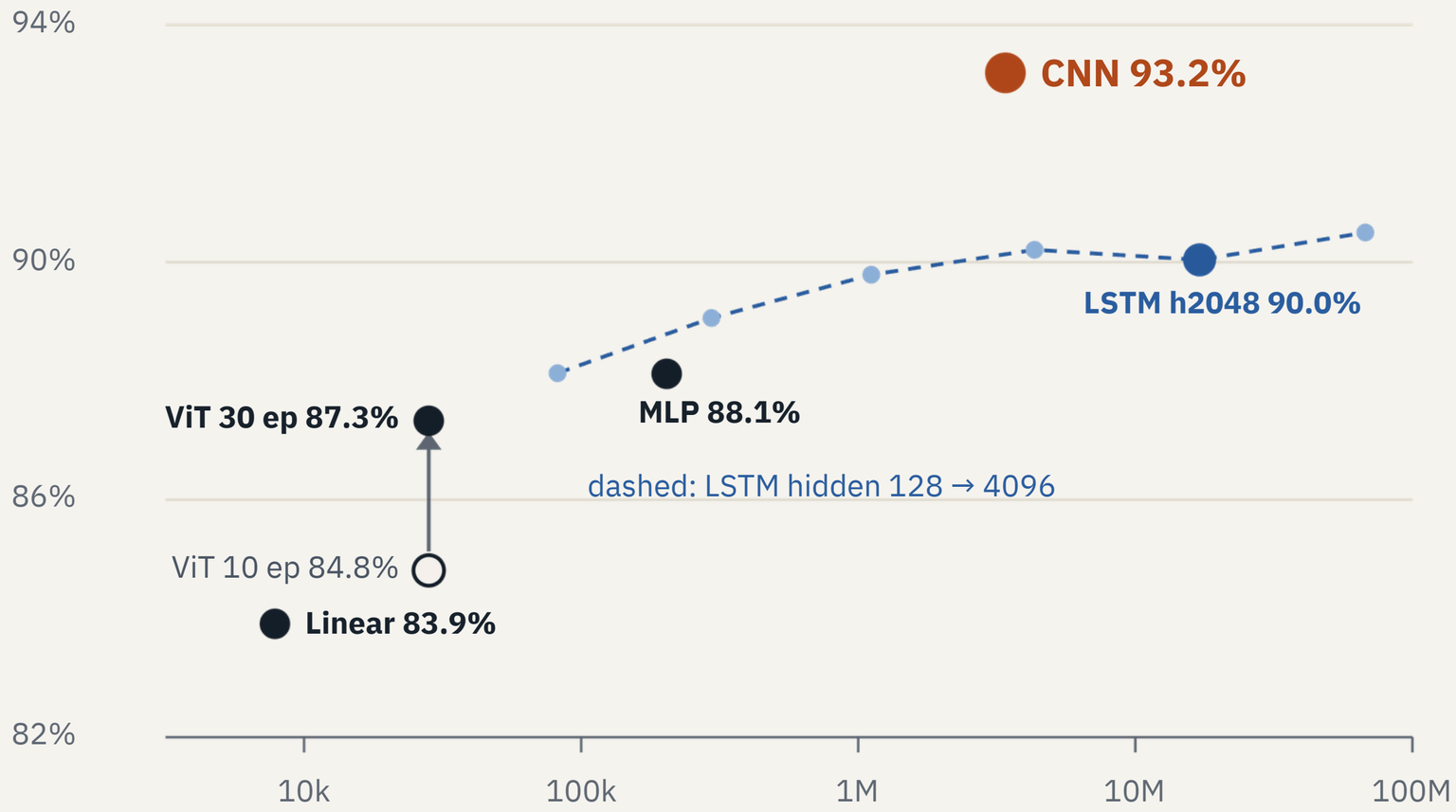
Val loss flat at ≈ 0.41 from epoch 5; train and val accuracy both stall near 86%. Capacity, not data, is the limit.

ViT · still learning

Slowest start (55% train acc at ep 1) and curves still climbing at ep 10 → it needs more epochs.

7 · TEST ACCURACY VS. TRAINABLE PARAMETERS (LOG SCALE)

Bigger is not better



7× fewer

ViT has 7.2× fewer parameters, yet its 30-epoch accuracy is only 0.79 percentage points below MLP

5× fewer

CNN has 5× fewer parameters but achieves 3.15 percentage points higher accuracy

823× params

Parameter count increases 823×, but accuracy improves only from 88.13% to 90.50%

7 · ABLATIONS

ViT wants time; LSTM does not need width

VIT · 10 → 30 EPOCHS

84.81% → 87.33%

Macro-F1 0.8492 → 0.8738

Best checkpoint is the last epoch: val F1 still rising (0.864 → 0.890) and val loss still falling (0.373 → 0.311). Under-trained, not over-fitted. Cost: 3× training time.

LSTM · HIDDEN SIZE SWEEP

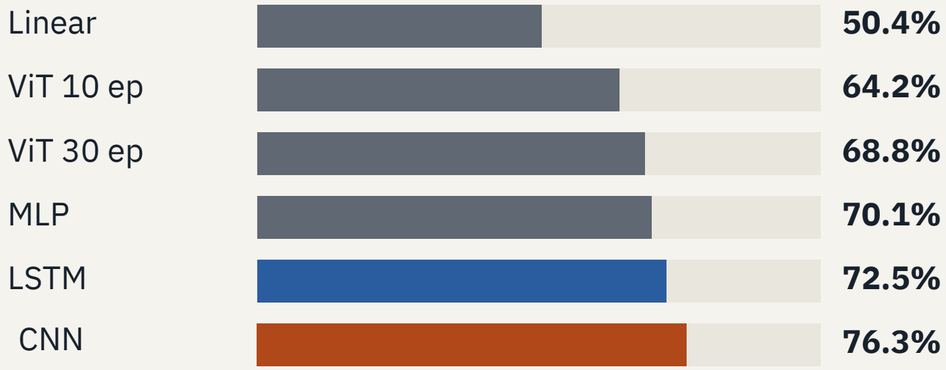
Hidden	Params	Test acc	Train
128	82,186	88.13%	68 s
256	295,434	89.06%	87 s
512	1,115,146	89.79%	65 s
1024 · sweet spot	4,327,434	90.21%	75 s
2048 · main run	17,043,466	90.04%	185 s
4096	67,641,354	90.50%	374 s

Gain saturates near 1024: the main 2048 model is 4× larger for no gain. Hidden size read from the logged parameter count; one run each (a 4096 repeat gave 90.37%).

8 · ERROR ANALYSIS · TEST SET

Errors cluster in look-alike garments

Shirt recall — worst class for every model



Other classes reach 89–99% recall with CNN.

306 images

are wrong for all five models — ambiguous or noisy labels no architecture fixed

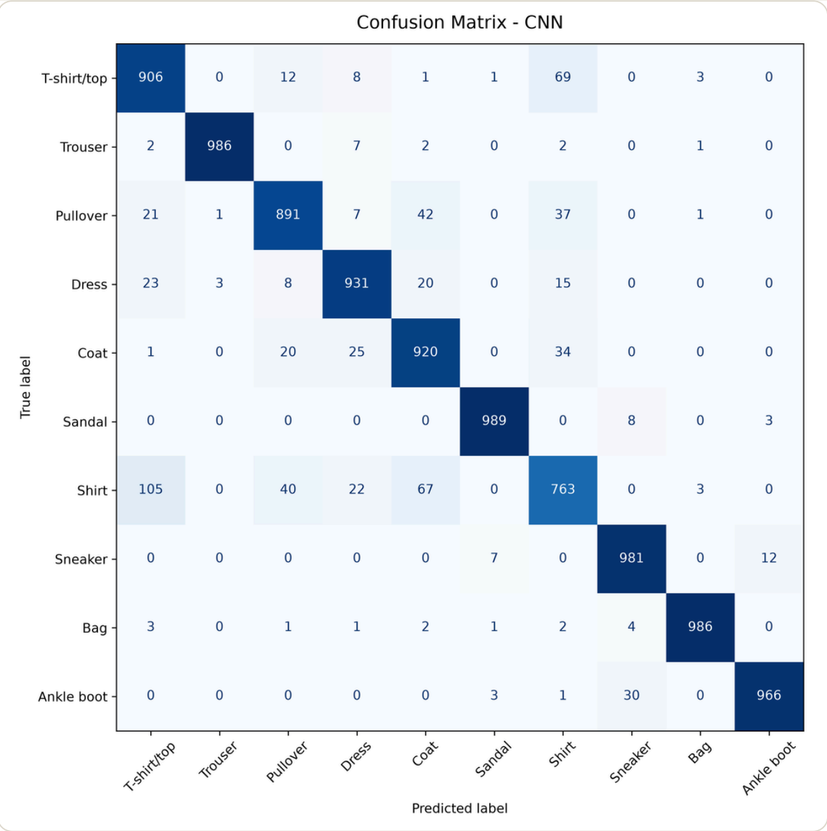
1 · Upper body: T-shirt, Pullover, Coat, Shirt

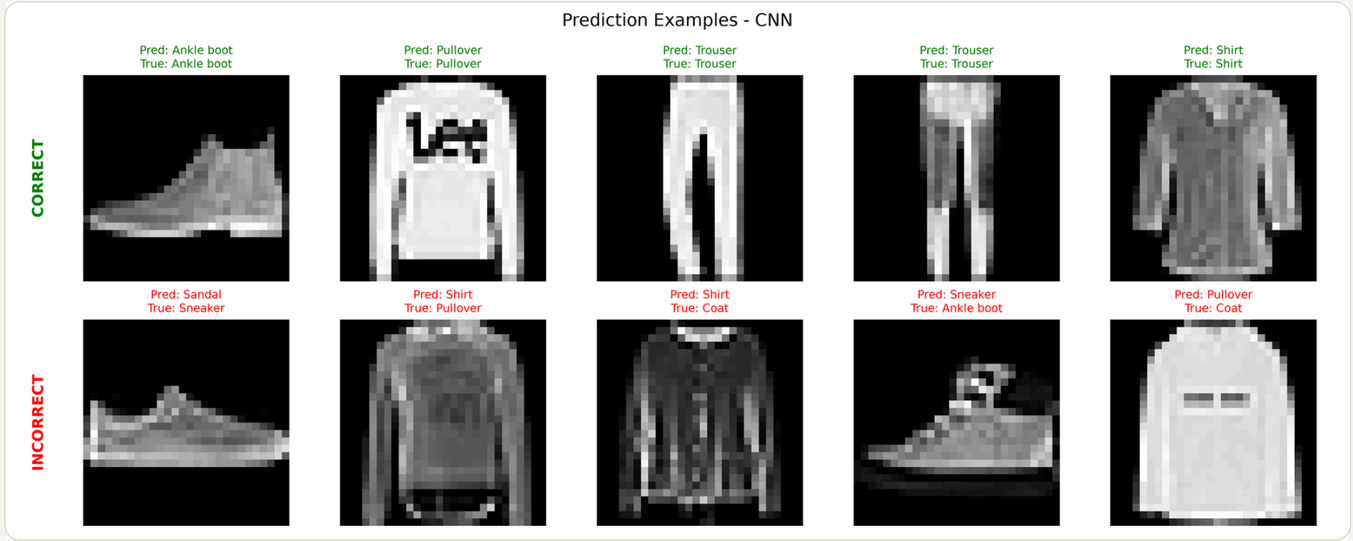
upper-body clothing, including T-shirt, Pullover, Coat and Shirt. This group accounts for approximately 66 percent of CNN's errors, or 449 out of 681 mistakes.

2 · Footwear: Sandal, Sneaker, Ankle boot

The Linear model makes 213 errors in this group, while CNN reduces this number to 63, approximately a 70 percent reduction.

What the best model still gets wrong





Hard but correct

The long-sleeve Shirt (top right) is right despite a pullover-like outline.

Plausible failures

Each mistake lands on a look-alike: no zipper → Pullover, low cut → Sandal.

9 · SHORT DEMO · REPRODUCE FROM THE REPO

Four commands from clone to confusion matrix

```
# 1. Environment setup (done before presentation)
pip install -r requirements.txt
# 2. Dataset preparation (done before presentation)
python src/data.py ; python src/eda.py
# 3. Training example (show execute)
python -m src.train --model linear

python -m src.train --model mlp

python -m src.train --model cnn

python -m src.train --model lstm

python -m src.train --model transformer

python -m src.train --model transformer --epochs 30
# 4 · best checkpoints → confusion matrices, examples
python -m src.evaluate
```

Per-epoch log

loss, accuracy, macro-F1 for train and val; epoch time

Artifacts written

checkpoints/cnn_ep10/best.pt, history CSV, a row in results/summary.csv

Re-evaluation matches

Seed 666 + saved split.npz; re-evaluating our checkpoints reproduces the table exactly

10 · CONCLUSION

Representation matters more than size

> Keeping 2-D locality (CNN) gives the best accuracy and F1. Flattening underfits; sequence views work but pay in parameters (LSTM) or in epochs (ViT).

Hypotheses

Exp.	Verdict	Evidence
EXP-1	Supported	in results/summary
EXP-2	Supported	CNN best at 93.19% (≥ 92%)
EXP-3	Supported	LSTM −3.15 points, 5× params, 1.9× train time

Limitations

One seed per model (no mean ± std); clean, centred 28×28 images; no augmentation or tuning; budgets not matched; inference time includes data loading.

Next steps

3+ seeds; matched-parameter comparison; augmentation and focal loss for Shirt; smaller CNN head; CIFAR-10.

DELIVERABLES & AI USAGE DISCLOSURE

Thank you — everything is reproducible

Code repository

github.com/phsuuuuuu/CO3133_sem261_groupRTX5090

Assignment page · report · slides

phsuuuuuu.github.io/
CO3133_sem261_groupRTX5090/assignment1.html

Checkpoints

[link to checkpoint folder] · or rebuild with the demo commands

Video

CO3133-Semester-261 – Group RTX5090 – Assignment 1

AI tools used (details in AI_USAGE.md)

ChatGPT · Dang Thanh Phu — website scaffold, DataLoader / argparse / CSV syntax, report explanations

Gemini 3.8 Flash · Cao Duc Thanh — matplotlib plots, LSTM debugging, ViT guidance with D2L, report structure

Claude Opus 5.5 · Cao Duc Thanh — drafting these slides from our report and logs

All numbers come from our own runs and were re-checked against the saved checkpoints.